TEDxDYPDPU Attendance Management System — Technical Requirements

Team, we need to build a TEDxDYPDPU Attendance Management System as a Next.js application, hosted on Vercel, with Google Sheets API as the attendance database.

Please follow the requirements below closely. You have flexibility in implementation decisions where something is not explicitly specified, but do not change the core flow or access rules.

1. TECH STACK

• Next.js + TypeScript

• Frontend + Backend within Next.js

• Hosted on Vercel

• Google Sheets API for reading/updating attendance

• Authentication will be hardcoded/config-based, not database-based

• Keep sensitive credentials/secrets in environment variables

• Use a clean, modular architecture so the system can be extended later

2. ACCESS / AUTHENTICATION

Only TE (Team Executive) members can access the application.

Login flow:

Open App → Select Domain → Select TE Name → Enter Password → Login

Domains, TE members, SE members and passwords should be maintained in configuration files, not scattered throughout the code.

Domains must follow this exact sequence:

1. Curation

2. Marketing

3. EPM

4. PRxEditing

5. Design

6. Tech

Each domain has its own TE members and SE members.

After successful login:

• TE can access attendance

• They can mark attendance only for members belonging to their respective domain

• They cannot view/edit attendance for other domains

• This restriction MUST be enforced in the backend as well, not only by hiding UI elements

3. ATTENDANCE SHEET FORMAT

Google Sheet must follow this standard structure:

Column A → Member Name

Column B → Domain

Column C onwards → Attendance dates

Example:

A = Name

B = Domain

C = 01/09/2026

D = 02/09/2026

E = 03/09/2026

Each date column contains:

• Present

• Absent

Use consistent formatting/casing throughout the application.

4. MEMBER ORDERING IN SHEET

The sheet must have a fixed, hardcoded order.

Exact domain sequence:

Curation TE

Curation SE

Marketing TE

Marketing SE

EPM TE

EPM SE

PRxEditing TE

PRxEditing SE

Design TE

Design SE

Tech TE

Tech SE

Within each domain, members should also follow the fixed order defined in configuration.

Do NOT dynamically reorder the sheet based on login/activity.

The number of TE and SE members should also be defined in configuration files so the application does not depend on manually changing frontend code.

5. ATTENDANCE UI

After login, show the logged-in TE's respective domain.

Display all members from that domain who are eligible for attendance marking.

Example:

Curation

Member Name Attendance

Member 1 [Present]

Member 2 [Absent]

Member 3 [Present]

Member 4 [Absent]

Use a toggle/button in front of each member's name.

The current attendance state should be extremely obvious.

The TE should be able to mark attendance for:

• SE members

• Other TE members

BUT only within their own domain.

6. TODAY'S DATE

Header should prominently display:

TEDxDYPDPU

01 September 2026

The date should be generated automatically based on the current date.

When attendance is submitted for today:

• If today's date column doesn't exist → create/add the date column

• If it already exists → update the existing cells

• NEVER create duplicate date columns

Consider timezone carefully. The application will be used in India, so use IST / Asia-Kolkata as the operational timezone.

7. SAVE / UPDATE BEHAVIOUR

Attendance changes should not unnecessarily create multiple API requests.

Preferred flow:

Change toggle locally → Save/Submit Attendance → Backend validates logged-in TE + domain → Backend updates corresponding Google Sheet cells

Handle:

• Google API failure

• Network failure

• Duplicate submissions

• Sheet structure mismatch

• Missing member

• Invalid date

• Unauthorized domain access

Show proper feedback:

“Attendance saved successfully”

or

“Unable to save attendance. Please try again.”

7A. DEFAULT STATE & LIVE SHEET DATA

• The default attendance state for any member who has not yet been marked for the selected date must be Absent.

• Do NOT assume that a member is Present unless the Google Sheet explicitly contains “Present” for that member on that date.

• Whenever the user opens/reopens the app, refreshes the page, logs in again, or navigates back to the Attendance section, the application should fetch the latest attendance data directly from Google Sheets.

• The Google Sheet should be treated as the source of truth for attendance.

• Do NOT rely only on previously cached/local frontend state.

• Example:

- TE marks Member 1 → Present

- Attendance is saved to Google Sheet

- TE closes the app

- TE opens the app again later

- App fetches the latest Google Sheet data

- Member 1 must appear as Present

• If the Sheet contains:

- “Present” → show Present

- “Absent” → show Absent

- Empty/missing value → treat and display as Absent

• On initial loading of the Attendance page, show a loading state while fetching the latest data.

• If fetching the Google Sheet fails, do not silently display potentially stale attendance data. Show an appropriate error and provide a Retry option.

• After successfully saving attendance, the UI should reflect the saved state and the Sheet should remain the source of truth.

• If multiple TEs from the same domain use the application, each should see the latest saved attendance when they open/refresh the app.

The expected behaviour is:

Open App

↓

Login

↓

Fetch latest data from Google Sheet

↓

Display live attendance

↓

Default missing/empty attendance = Absent

↓

TE changes toggles

↓

Save

↓

Update Google Sheet

↓

Refresh/reopen app

↓

Fetch latest data again

↓

Display current Sheet state

IMPORTANT: Google Sheets is the source of truth. Frontend state is only temporary UI state and must never override or replace the actual Sheet data.

8. HEADER / HAMBURGER MENU

Header should contain:

TEDxDYPDPU + Today's Date

Hamburger menu should contain:

[Logged-in Member Name]

Domain: Curation

Role: TE

Attendance

Analytics

Change Password

Logout

Logout should clear the current authenticated session.

9. CHANGE PASSWORD

Provide a Change Password section.

Flow:

Current Password

New Password

Confirm New Password

↓

Change

Current password must be validated before changing it.

The new password should replace the existing password for that TE.

Important: because this is intentionally a hardcoded/config-based authentication system, decide on a sensible persistence mechanism for changed passwords.

Do NOT assume that modifying a deployed source/config file at runtime is possible.

Prefer an architecture where initial credentials are configuration/env based, while changed passwords can be persisted securely if required.

NEVER expose passwords to the frontend.

10. ANALYTICS — OPTIONAL

Add an Analytics section if practical.

It should provide a visual representation of attendance for the logged-in TE's domain.

Possible analytics:

• Total members
• Present today
• Absent today
• Attendance percentage
• Member-wise attendance percentage
• Date-wise attendance
• Present/Absent trend

Example:

Curation Attendance

Total Members: 12

Present Today: 10

Absent Today: 2

Attendance: 83.3%

Member 1 → 100%

Member 2 → 80%

Member 3 → 90%

...

Analytics MUST follow the same domain-level access restriction.

A Curation TE must never receive Marketing/Design/Tech attendance data through the API.

11. DESIGN

Use a TEDx-style minimal visual language.

Primary colors:

• Black
• White
• TEDx-style Red
• Dark/light shades of these

The application should feel:

• Premium
• Minimal
• Modern
• Fast
• Professional
• Mobile-friendly

Avoid unnecessary animations and visual clutter.

Make the attendance action extremely easy because this will primarily be used quickly during events.

12. SECURITY REQUIREMENTS

Even though authentication is hardcoded/config-based, treat security seriously.

Implement:

• Server-side authentication
• Secure session handling
• No passwords in client-side JavaScript
• No Google service-account credentials exposed to frontend
• Google Sheets API calls only from backend/server
• Backend authorization checks
• Domain-level authorization
• Input validation
• Rate limiting / reasonable protection against brute-force login attempts
• Secure logout
• Environment variables for Google credentials

DO NOT rely on frontend restrictions for security.

Example:

Hiding Marketing from a Curation TE is NOT sufficient.

Backend must reject a request attempting to access Marketing data.

13. GOOGLE SHEETS API

Create the complete integration.

Architecture:

Next.js Frontend

↓

Next.js API / Server Actions

↓

Google Sheets API

↓

Google Sheet

Google credentials and Sheet ID should come from environment variables.

Create reusable functions/services such as:

```
getSheetData()
getMembers()
getTodayColumn()
createDateColumn()
getDomainMembers()
getAttendance()
updateAttendance()
```

Keep Google Sheets-specific logic isolated from UI logic.

14. CONFIGURATION

Create a centralized configuration structure.

Something conceptually like:

```
config/
```

members.ts

domains.ts

auth.ts

It should contain:

• Domain sequence

• TE members

• SE members

• Fixed member ordering

• Initial credentials/configuration

• Roles

• Other required constants

Do NOT hardcode member names/passwords throughout components.

15. IMPORTANT EDGE CASES

Explicitly handle:

• User selects wrong domain for a TE

• Wrong password

• Empty password

• Non-existent TE

• Unauthorized API request

• TE attempting another domain

• Google Sheets API unavailable

• Internet disconnected

• Today's column already exists

• Today's column doesn't exist

• Duplicate attendance submission

• Sheet rows missing

• Member exists in config but not sheet

• Member exists in sheet but not config

• Invalid sheet format

• Multiple TEs editing attendance simultaneously

• Session expiry

• Logout

• Password change failure

• Password confirmation mismatch

• Browser refresh after login

• Mobile screen sizes

For simultaneous edits, avoid blindly overwriting unrelated members' attendance.

16. EXPECTED PROJECT STRUCTURE

Use a clean structure along these lines:

```
app/
login/
attendance/
```

analytics/

api/

auth/

attendance/

analytics/

password/

components/

Header/

HamburgerMenu/

AttendanceList/

AttendanceToggle/

Analytics/

UI/

lib/

google-sheets/

auth/

session/

validation/

config/

domains.ts

members.ts

auth.ts

types/

attendance.ts

member.ts

auth.ts

You can modify the structure if you have a better Next.js architecture, but maintain clear separation between:

UI → API → Business Logic → Google Sheets

17. DELIVERABLES

Please deliver:

1. Complete Next.js + TypeScript application

2. Responsive frontend

3. Login/authentication

4. TE domain authorization

5. Attendance marking

6. Google Sheets API integration

7. Automatic date-column handling

8. Change-password functionality

9. Logout/session handling

10. Optional analytics dashboard

11. Proper error handling

12. Environment-variable configuration

13. Deployment-ready Vercel configuration

14. README with complete setup instructions

README should explain:

1. How to install

2. Environment variables

3. Google Cloud setup

4. Google Sheets API setup

5. Service account / permissions

6. Sheet format

7. How to configure TE/SE members

8. How to configure initial passwords

9. How authentication works

10. How to deploy to Vercel

11. How to add/remove members

12. Known limitations

FINAL REQUIREMENT

Before considering the implementation complete, test the complete flow using different TE users.

Minimum test cases:

Curation TE → Curation attendance → SUCCESS

Curation TE → Marketing attendance → BLOCKED

Marketing TE → Marketing attendance → SUCCESS

Marketing TE → Curation attendance → BLOCKED

Wrong password → BLOCKED

Non-TE → BLOCKED

Attendance today → Correct date column

Attendance tomorrow → New date column

Existing date → Update existing column

The most important principle:

EVERY TE CAN MANAGE ATTENDANCE, BUT ONLY FOR THEIR OWN DOMAIN.

Build the entire system around this authorization rule rather than treating it as only a UI restriction.

Please also make sensible technical decisions wherever required, but keep the above functionality and flow unchanged.